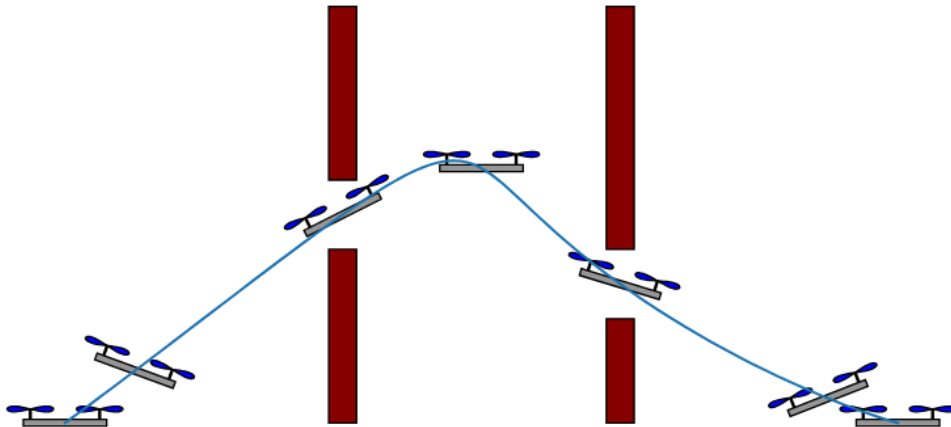


How to Generate Optimal Constrained Trajectories:

How to use Optimal Constrained Trajectory MATLAB Calculator

Luke Kizah (lgkiziah@ncsu.edu)
Abhinav Indukuri (ainduku@ncsu.edu)



1. Introduction

Autonomous systems are actively being deployed into real-world applications, where the ability to efficiently and safely navigate through dynamic environments is essential. Whether it's a drone navigating through a forest, or a spacecraft navigating through asteroids, all these systems have limitations, tolerances, and obstacles. Optimal constrained trajectories are utilized by these systems to accomplish this.

Optimal constrained trajectory optimization involves utilizing a mathematical framework to take in defined variables and constraints of a system, weight them based on importance, and compute the best path for the system from a particular state. The result is a trajectory that may not intuitively be the best option, but is based on the system's dynamics and limitations. As autonomous systems become more complex and capable, the ability to compute these trajectories quickly and reliably is essential.

The following instructions cover a MATLAB tool that has been developed, as a part of a research effort, to determine these optimal constrained trajectories for various systems. For the sake of these instructions, a cart-pendulum example is used to demonstrate how to use the MATLAB app. These instructions will cover what information is required, what each field means, and how to correctly populate each field before running the program. You can download the code necessary for running the example program [here](#).

2. Background

Before we dive into the app, I will briefly explain some important terms related to optimal constrained trajectories.

2.1 What is a Trajectory and a State?

A trajectory is the time history of a system's state. Over time, a trajectory traces how a system's states evolve from the initial condition to the final or present condition. A state vector defines the system at a given moment and consists of a set of state variables. For a rocket it might be the rocket's position, velocity, and orientation.

2.2 What is an Optimal Trajectory?

Trajectory optimization determines the best possible way a system can move out of all the possible ways the system could move from one point to another. This "best trajectory" is determined based on a cost function, a mathematical expression that is used to evaluate each trajectory. This cost function can be tuned based on what you are trying to optimize for, whether it is time, or distance, or energy used. The optimizer works to determine the sequence that has the lowest cost value.

2.3 What are Constraints?

Real-world systems still have to obey the laws of physics and the conditions of its use case, so they cannot move freely. Constraints are the defined rules that the trajectory must obey at all times. The MATLAB application uses three types of constraints, input bounds, state bounds, and terminal state constraints. Input bounds are the limits of what can be signalized, like a rocket can only produce a certain amount of thrust. The state bounds are limits on where the system is allowed to be throughout the trajectory, like a rocket cannot have an altitude below 0. Lastly, terminal state constraints are requirements set for where the system must be at the end of the trajectory.

The solver takes these constraints and then must find a trajectory that minimizes cost and satisfies these constraints. This is what is known as an optimal constrained trajectory.

2.4 Additional Information

There is a lot more that mathematically goes into the process of determining an optimal constrained trajectory. More information on the underlying theory can be found in this folder:

 [An Accessible Gateway to Trajectory Optimization](#)

Instructions

3. Tab 1 - Dynamics

When the MATLAB code is run and the app window opens, you will first see the Dynamics tab. This is the tab where the mathematical descriptions of the system's physics are entered. Once the dynamics are entered, they must be confirmed before the following tabs can be populated and used. This process applies to the Constraints, and Objective tabs as well. The tab collects the number of state variables, number of control inputs, the equations of motion (EOMs), physical constants, and the system's initial state.

4.1 Dimension Fields

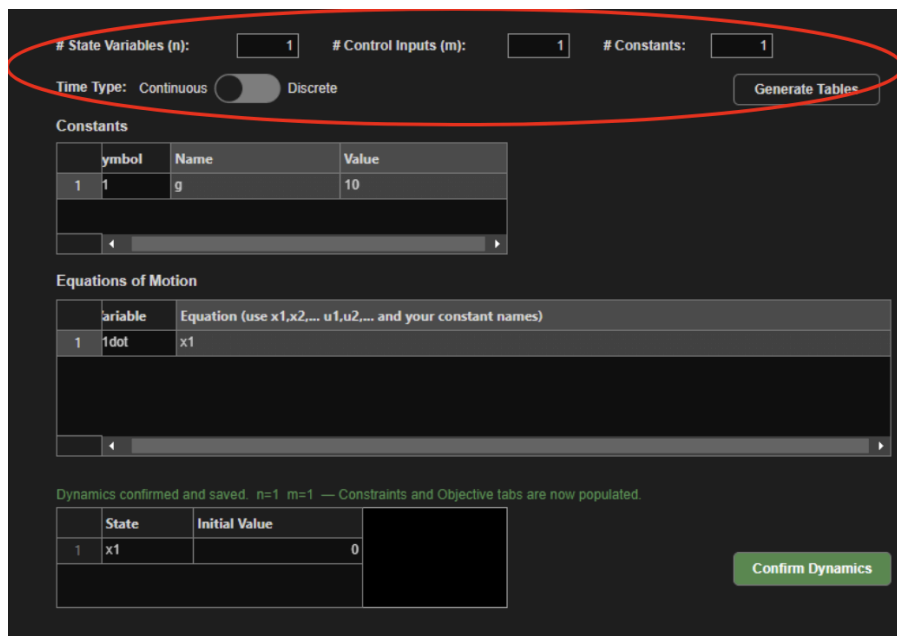
At the top of the tab, there are three fields that define dimensions moving forward. These fields are:

- # State Variables (n) - the number of variables that define the system's state (e.g., position, velocity, etc.)
- # Control Inputs (m) - the number of control signals applied to the system (e.g., torque, thrust, etc)
- # Constants - the number of fixed variables the EOMs reference (e.g., gravity, mass, etc)

Once these values are entered, click the “Generate Tables” button, and the Constants, EOMs, and Initial State tables will populate, each sized to the values put into the dimension fields.

4.2 Time Type Toggle

Right below the dimension fields is a switch labeled “Time Type”, where you can specify whether the EOMs are written in continuous or discrete time. Continuous EOM are in the form of the time rate of change of state as a function of current state and input. The solver will automatically translate these equations into discrete versions. EOMs in discrete time describe the state one step ahead (e.g., $x1_{k+1} = x1_k + x2_k \cdot dt$).



The screenshot shows a software interface for configuring a dynamical system. At the top, there are three input fields: "# State Variables (n):" with the value 1, "# Control Inputs (m):" with the value 1, and "# Constants:" with the value 1. Below these is a "Time Type" toggle switch set to "Continuous". A red oval highlights the top section containing these fields and a "Generate Tables" button. Below the toggle, there are three tables: "Constants", "Equations of Motion", and "Initial State".

	symbol	Name	Value
1	c1	g	10

	variable	Equation (use x1,x2,... u1,u2,... and your constant names)
1	1dot	x1

	State	Initial Value
1	x1	0

At the bottom right, there is a green "Confirm Dynamics" button. A status message at the bottom reads: "Dynamics confirmed and saved. n=1 m=1 — Constraints and Objective tabs are now populated."

Figure 1: Dimension Field and Time Toggle Location

4.3 Constants Table

Each row of the Constants table corresponds with one physical constant. The symbol column is automatically filled (c1, c2, etc.) and cannot be changed. However, the other two columns, Name and Value, must be filled out. The Name column is the variable name, as it would appear in the EOMs. The Value column is simply the value associated with this constant.

State Variables (n): # Control Inputs (m): # Constants:

Time Type: Continuous ☒ Discrete ☐

[Generate Tables](#)

Constants

	Symbol	Name	Value
1	1	g	10

Equations of Motion

	Variable	Equation (use x1,x2,... u1,u2,... and your constant names)
1	1dot	x1

Dynamics confirmed and saved. n=1 m=1 — Constraints and Objective tabs are now populated.

	State	Initial Value
1	x1	0

[Confirm Dynamics](#)

Figure 2: Constants Table Location

4.4 Equations of Motion Table

Each row of the EOMs table corresponds with a state variable. The Variable column is automatically filled (x1dot, x2dot, etc.). Each variable will need an equation of motion put into the equations tab in standard MATLAB syntax. Each state variable is referenced as x1, x2, etc. Control inputs are referenced as u1, u2, etc. Lastly, constants are referenced by the name that the user has defined in the Names column of the Constants table.

State Variables (n): # Control Inputs (m): # Constants:

Time Type: Continuous ☒ Discrete ☐

[Generate Tables](#)

Constants

	Symbol	Name	Value
1	1	g	10

Equations of Motion

	Variable	Equation (use x1,x2,... u1,u2,... and your constant names)
1	1dot	x1

Dynamics confirmed and saved. n=1 m=1 — Constraints and Objective tabs are now populated.

	State	Initial Value
1	x1	0

[Confirm Dynamics](#)

Figure 3: EOMs Table Location

4.5 Initial State Table

Lastly, the Initial State table sets the initial values of the state variables. Each row corresponds to a state variable (x_1 , x_2 , etc.). The Initial Value column is where the initial values can be entered for the corresponding state variable. They are automatically filled with zeros, and can be changed to the actual initial value.

The screenshot shows a configuration interface for a control system. At the top, there are input fields for '# State Variables (n):' (value 1), '# Control Inputs (m):' (value 1), and '# Constants:' (value 1). Below these is a 'Time Type' section with 'Continuous' selected and 'Discrete' as an option. A 'Generate Tables' button is on the right. The 'Constants' section contains a table with columns 'symbol', 'Name', and 'Value'. The first row has '1' in 'symbol', 'g' in 'Name', and '10' in 'Value'. Below this is a horizontal scrollbar. The 'Equations of Motion' section contains a table with columns 'variable' and 'Equation (use x1,x2,... u1,u2,... and your constant names)'. The first row has '1' in 'variable' and 'x1' in 'Equation'. Below this is another horizontal scrollbar. A status message in green text reads: 'Dynamics confirmed and saved. n=1 m=1 — Constraints and Objective tabs are now populated.' Below the message is the 'Initial State Table' with columns 'State' and 'Initial Value'. The first row has 'x1' in 'State' and '0' in 'Initial Value'. This table is circled in red. A 'Confirm Dynamics' button is located at the bottom right.

symbol	Name	Value
1	g	10

variable	Equation (use x1,x2,... u1,u2,... and your constant names)
1	x1

Dynamics confirmed and saved. n=1 m=1 — Constraints and Objective tabs are now populated.

State	Initial Value
x1	0

Confirm Dynamics

Figure 4: Initial State Table Location

4.6 Confirming Dynamics

After all the fields have been populated, click the “Confirm Dynamics” button. This will validate the inputs for completeness, update the variables that are relevant to the solver, and automatically populate the Constraints and Objective tabs. A status message is used to tell the user if the dynamics have been confirmed or which field is incomplete.

State Variables (n):
Control Inputs (m):
Constants:

Time Type: ☒ Continuous ☐ Discrete

Generate Tables

Constants

	Symbol	Name	Value
1	1	g	10

Equations of Motion

	Variable	Equation (use x1,x2,... u1,u2,... and your constant names)
1	1dot	x1

Dynamics confirmed and saved. n=1 m=1 — Constraints and Objective tabs are now populated.

	State	Initial Value
1	x1	0

Confirm Dynamics

Figure 4: Confirm Dynamics Button Location

4.6 Example

For our cart-pendulum example, this is how the dynamics tab would be filled out before moving to the constraints tab. For ease of use, acceleration equations can be copied from below.

$$x2dot = (u1 - c * x2 - m * l * x4^2 * \sin(x3) + m * g * \sin(x3) * \cos(x3)) / (M + m * \sin(x3)^2)$$

$$x4dot = (M * g * \sin(x3) + m * g * \sin(x3) - m * l * (x4)^2 * \sin(x3) * \cos(x3) + u1 * \cos(x3) - c * x2 * \cos(x3)) / (M * l + m * l * \sin(x3)^2)$$

Dynamics
Constraints
Objective
Solver
Compute
Example

State Variables (n): 4
Control Inputs (m): 1
Constants: 5

Time Type: Continuous ☒ Discrete ☐
Generate Tables

Constants

	Symbol	Name	Value
1	c1	l	1
2	c2	m	1
3	c3	M	2
4	c4	c	.5
5	c5	g	10

Equations of Motion

	Variable	Equation (use x1,x2,..., u1,u2,... and your constant names)
1	x1dot	x2
2	x2dot	$(u1 - c^*x2 - m^*l^*x4^2*\sin(x3) + m^*g*\sin(x3)*\cos(x3))/(M + m*\sin(x3)^2)$
3	x3dot	x4
4	x4dot	$(M^*g*\sin(x3) + m^*g*\sin(x3) - m^*l^*(x4)^2*\sin(x3)*\cos(x3) + u1*\cos(x3) - c^*x2*\cos(x3))/(M^*l + m^*l*\sin(x3)^2)$

Dynamics confirmed and saved. n=4 m=1 — Constraints and Objective tabs are now populated.

	State	Initial Value
1	x1	0.5000
2	x2	0.1000
3	x3	0.4000
4	x4	0

Confirm Dynamics

Figure 5: Dynamics Tab of Cart-Pendulum Example

4. Tab 2 - Constraints

The Constraints tab defines the physical limitations of the system that must be obeyed throughout the trajectory. There are up to 3 tables in this tab, with the first two being automatically populated with the values of -Inf and Inf (-Infinity/Infinity). These fields will autopopulate and can be edited after the Dynamics have been confirmed in the previous tab.

4.1 Input Bounds

This is the first table where each row corresponds with a control input. Each row has a max and min column where the maximum and minimum allowable values during any time step can be defined. If a certain control input is unconstrained, the default values of -Inf and Inf can be left how they are.

Input Bounds
Limits on control inputs at every timestep. Leave as -Inf / Inf if unconstrained.

	Input	Min	Max
1	u1	-Inf	Inf

State Bounds
Limits on states at every timestep. Leave as -Inf / Inf if unconstrained.

	State	Min	Max
1	x1	-Inf	Inf

☒ **Enable Terminal State Constraint**
Forces trajectory to end at exact values. When active, Qf in Objective tab is disabled.

	State	Required Final Value
1	x1	

Fill in bounds then click Confirm Constraints.

Confirm Constraints

Figure 6: Input Bounds Table Location

4.2 State Bounds

This is the next table where each row corresponds to a state variable. Just like the last field, each row has a max and min column, but this is where the maximum and minimum values of the state variables can be at any point in the trajectory. Once again, if the state variables have no limitations then these fields can be left as the default values, -Inf and Inf.

Input Bounds
Limits on control inputs at every timestep. Leave as -Inf / Inf if unconstrained.

	Input	Min	Max
1	u1	-Inf	Inf

State Bounds
Limits on states at every timestep. Leave as -Inf / Inf if unconstrained.

	State	Min	Max
1	x1	-Inf	Inf

☒ **Enable Terminal State Constraint**
Forces trajectory to end at exact values. When active, Qf in Objective tab is disabled.

	State	Required Final Value
1	x1	

Fill in bounds then click Confirm Constraints.

Confirm Constraints

Figure 7: State Bounds Table Location

4.3 Terminal State Constraints

There is a checkbox that can be activated if the user would like to include Terminal State Constraints. This will result in a new table being generated with each row corresponding to a state variable. There is a column named “Required Final Value” where the desired final value can be entered for the corresponding state variable. The solver will then solve for a trajectory that must hold these final values.

When the Terminal State Constraints checkbox is checked and enabled, the Qf matrix in the objective tab will automatically be disabled to remove redundancy in the solver. Once all constraints are set, the user can click the “Confirm Constraints” button to save these values.

Input Bounds
Limits on control inputs at every timestep. Leave as -Inf / Inf if unconstrained.

	Input	Min	Max
1	u1	-Inf	Inf

State Bounds
Limits on states at every timestep. Leave as -Inf / Inf if unconstrained.

	State	Min	Max
1	x1	-Inf	Inf

☒ **Enable Terminal State Constraint**
Forces trajectory to end at exact values. When active, Qf in Objective tab is disabled.

	State	Required Final Value
1	x1	

Fill in bounds then click Confirm Constraints.

Confirm Constraints

Figure 8: Terminal State Constraints Table Location

4.4 Example

For our cart-pendulum example, this is how the Constraints tab would be filled out.

Dynamics

Constraints

Objective

Solver

Compute

Example

Input Bounds

Limits on control inputs at every timestep. Leave as -Inf / Inf if unconstrained.

	Input	Min	Max
1	u1	-20	20

State Bounds

Limits on states at every timestep. Leave as -Inf / Inf if unconstrained.

	State	Min	Max
1	x1	-2	2
2	x2	-5	5
3	x3	-1	1
4	x4	-2	2

☒ Enable Terminal State Constraint

Forces trajectory to end at exact values. When active, Qf in Objective tab is disabled.

	State	Required Final Value
1	x1	0
2	x2	0
3	x3	0
4	x4	0

Constraints confirmed and saved to workspace.

Confirm Constraints

Figure 9: Constraints Tab of Cart-Pendulum Example

5. Tab 3 - Objective

The Objective tab defines the cost function the solver works to minimize. This cost function follows the following formula:

$$J = \sum (x^T Q x + u^T R u) + x_N^T Q_f x_N$$

This sums, over all time steps, a penalty on the state (weighted by Q) and on control effort (weighted by R), plus a terminal penalty on the final state (weighted by Qf). Each matrix is configured in different tables, with all three matrices having a default diagonal weight of 1 and can be edited.

5.1 Q - Running State Cost

Q is an $n \times n$ diagonal matrix that penalizes the deviation of the state variable throughout the trajectory. Each row of the table corresponds to a state variable with an editable “Diagonal Weight” column. This will let the user set larger values to state variables that are more important.

☒ **Enable Running State Cost Q ($n \times n$)**
Penalizes state deviation from zero throughout the trajectory. Diagonal weights only.

	State	Diagonal Weight
1	x1	1

☒ **Enable Terminal Cost Qf ($n \times n$)**
Penalizes state deviation from zero at the final time only.

	State	Diagonal Weight
1	x1	1

☒ **Enable Control Cost R ($m \times m$)**
Penalizes large control inputs. Keep > 0 to avoid numerical blow-up.

	Input	Diagonal Weight
1	u1	1

Set weights then click Confirm Objective.

Confirm Objective

Figure 10: Running State Cost Table Location

5.2 Qf - Terminal State Cost

Qf is an $n \times n$ diagonal matrix that penalizes state deviation at the final time only. This functions identically to the Q term but only applies to the end of the trajectory. If Qf is disabled (It will be disabled if Terminal State Constraints are enabled) by unchecking the box, the solver sets $Qf = 10Q$ to aid the solver.

☒ **Enable Running State Cost Q ($n \times n$)**
Penalizes state deviation from zero throughout the trajectory. Diagonal weights only.

	State	Diagonal Weight
1	x1	1

☒ **Enable Terminal Cost Qf ($n \times n$)**
Penalizes state deviation from zero at the final time only.

	State	Diagonal Weight
1	x1	1

☒ **Enable Control Cost R ($m \times m$)**
Penalizes large control inputs. Keep > 0 to avoid numerical blow-up.

	Input	Diagonal Weight
1	u1	1

Set weights then click Confirm Objective.

Confirm Objective

Figure 11: Terminal State Cost Table Location

5.3 R - Control Cost

R is an $m \times m$ diagonal matrix that penalizes the use of large control inputs. A larger R weight would have the solver find a trajectory that is more control effort efficient.

☒ **Enable Running State Cost Q (n x n)**
Penalizes state deviation from zero throughout the trajectory. Diagonal weights only.

	State	Diagonal Weight
1	x1	1

☒ **Enable Terminal Cost Qf (n x n)**
Penalizes state deviation from zero at the final time only.

	State	Diagonal Weight
1	x1	1

☒ **Enable Control Cost R (m x n)**
Penalizes large control inputs. Keep > 0 to avoid numerical blow-up.

	Input	Diagonal Weight
1	u1	1

Set weights then click Confirm Objective.

Confirm Objective

Figure 12: Control Cost Table Location

5.4 Enable/Disable Matrices

Each matrix panel has a checkbox that includes or excludes the respective matrix from the cost function. Mathematically, it sets the Q and R diagonal matrices to 0 and the Qf matrix to 10Q.

Once the weight matrices are defined, the user can click the “Confirm Objective” button to save the cost weights.

5.5 Example

For our cart-pendulum example, this is how the Objective tab would be filled out.

Dynamics
Constraints
Objective
Solver
Compute
Example

☒ **Enable Running State Cost Q ($n \times n$)**
Penalizes state deviation from zero throughout the trajectory. Diagonal weights only.

	State	Diagonal Weight
1	x1	2
2	x2	2
3	x3	2
4	x4	2

☐ **Enable Terminal Cost Q_f ($n \times n$)** Disabled: terminal state constraint is active.
Penalizes state deviation from zero at the final time only.

	State	Diagonal Weight
1	x1	1
2	x2	1
3	x3	1
4	x4	1

☒ **Enable Control Cost R ($m \times m$)**
Penalizes large control inputs. Keep > 0 to avoid numerical blow-up.

	Input	Diagonal Weight
1	u1	1.0000e-03

Objective confirmed and saved to workspace.

Confirm Objective

Figure 13: Objective Tab of Cart-Pendulum Example

6. Tab 4 - Solver

The Solver tab configures the time parameters of the application and exports values of the system to the MATLAB workspace, so the solver can then solve for the best trajectory.

6.1 Final Time

The Final Time, T_f , field sets the total time duration of the trajectory. The solver will find the optimal control sequence over the specified time interval.

6.2 Resolution (N Timesteps)

The Resolution field sets the number of discrete time steps, N . A larger resolution value means more time steps so a finer resolution and greater accuracy, but a longer computation time. The time steps size, dt , is computed automatically.

6.3 Exporting the Problem

Once the Final Time and Resolution are defined, the user can click the “Export Problem to Workplace” button to bundle all the user inputs into a MATLAB data file that saves to the workspace. This button will work and export the data even if some tabs have not been confirmed, but a warning message will prompt the user. It is recommended that all tabs are confirmed before exporting. After the data is exported, the user can go to the Compute tab to run the solver.

6.4 Example

For our cart-pendulum example, this is how the Solver tab would be filled out.

The screenshot shows the 'Solver' tab of a software interface. At the top, there is a horizontal menu bar with tabs: 'Dynamics', 'Constraints', 'Objective', 'Solver' (which is active), 'Compute', and 'Example'. Below the menu, the 'Final Time T (s):' field is set to 10, and the 'Resolution (N timesteps):' field is set to 100. A large blue button labeled 'EXPORT PROBLEM TO WORKSPACE' is centered below these fields. Underneath the button, a small text line states: 'Saves everything as TrajOptProblem in the MATLAB workspace for the solver script.' Below that, a green status message reads: 'Success: TrajOptProblem exported to workspace. Run your solver script now.'

Figure 14: Solver Tab of Cart-Pendulum Example

7. Tab 5 - Compute

The Compute tab runs the NMPC solver and displays the results. The system data must have been exported from the Solver tab before the solver can be used.

7.1 Running the Solver

Click the Run NMPC button to run the solver. The solver will run in the background while keeping the application responsive. A status label next to the button will change to “Running NMPC...” while the solver is active.

7.2 Solver Log

A text area below the run button displays live log messages as the solver progresses. As the engine solves subproblems towards convergence on the solution it sends messages to the log. These messages are meant to diagnose whether the solver is making progress or encountering errors towards reaching convergence criteria.

When the solver finishes, the logs will display a final message saying “NMPC finished” and the status label changes to “Complete”. If the solver fails at any iteration, the log will report a failure message and the status label will read “Solver failed”.

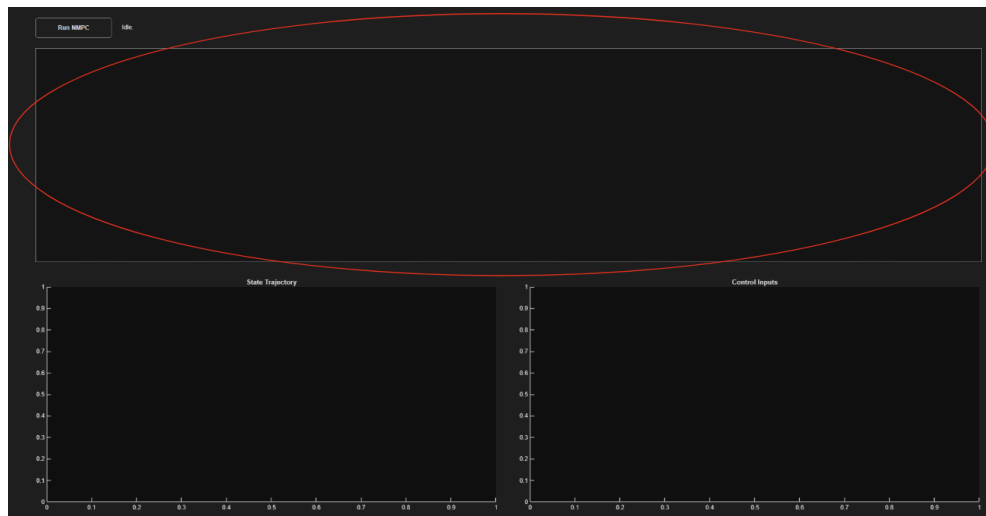


Figure 15: Solver Log Location

7.3 Result Plot

After the trajectory is successfully solved, two plots will be displayed at the bottom of the tab:

- States - Plots all state variables over time. Each state is drawn as a separate labeled line.
- Inputs - Plots all control inputs over time. This plot will be displayed as a step plot to reflect the step based control authority over the N time steps specified.

These plots will only appear after the solver computes, and are not updated live during the computation.

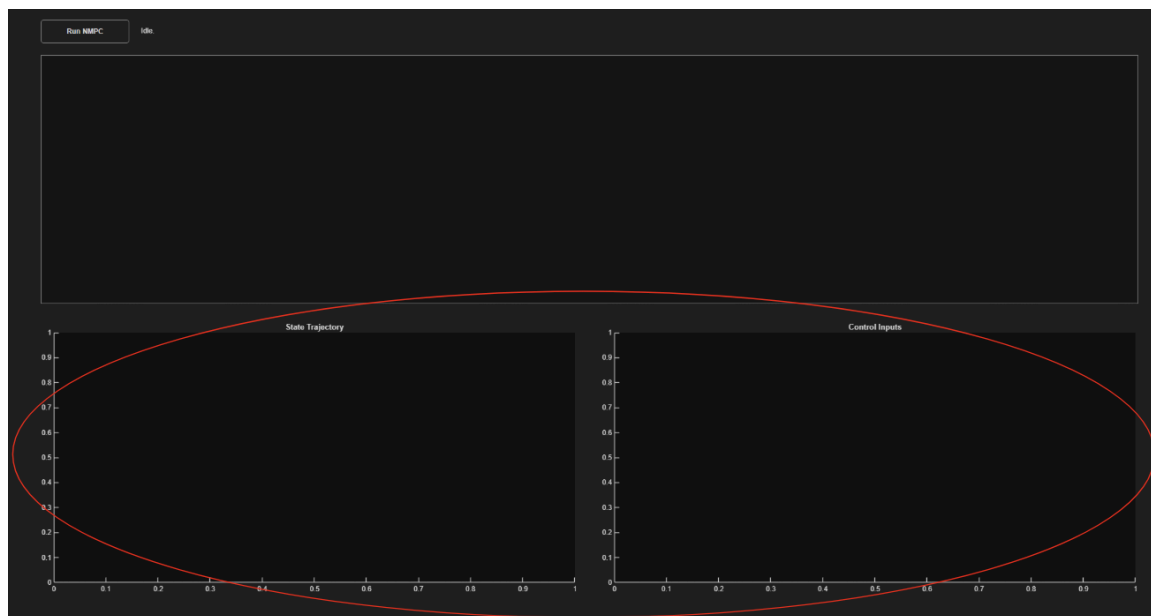


Figure 16: Result Plot Location

7.4 Example

For our cart-pendulum example, this is how the Solver tab would populate after running.

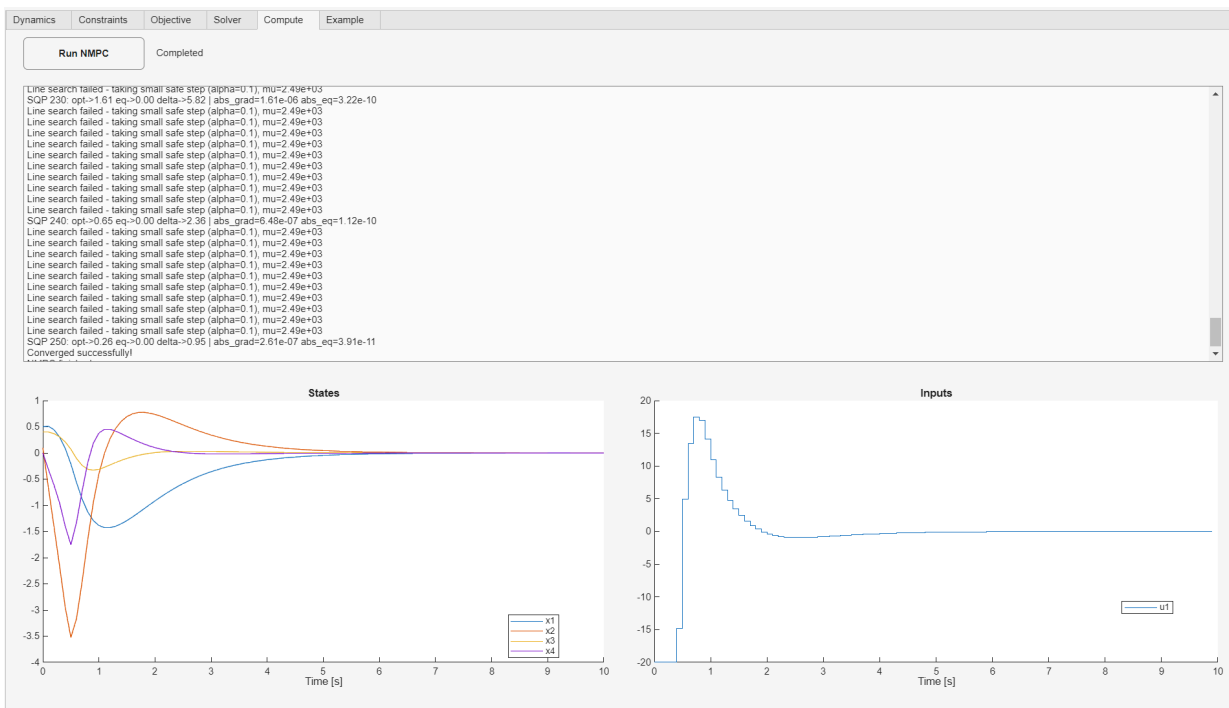


Figure 16: Compute Tab of Cart-Pendulum Example

8. Tab 6 - Example

For the sake of these instructions, we have added an Example tab, to verify the solution's and the application's accuracy. With a pre-computed MATLAB data file for the cart-pendulum example. The Example tab will compare the computed results with the pre-determined solution of this system to confirm the solver's results. Additionally, the tab has an animation of the cart-pendulum example.

8.1 Validation Check

Clicking the Evaluate Example Check Case button will compute the total error between the computed and reference solution. The result is displayed next to the button, with a Pass and Fail result that is determined by whether the total error is within the tolerance. This check is only useful to check that the example was exactly followed.

8.2 Cart-Pendulum Animation

Clicking the Run Animation button will play an animation of the cart-pendulum system moving through the computed optimal constrained trajectory. Each frame of the animation will correspond to one time step of the trajectory. Once again, this animation is specific to this system and is not generalized to any arbitrary problem entered by the user.

8.3 Example

For our cart-pendulum example, this is how the Example tab would populate.

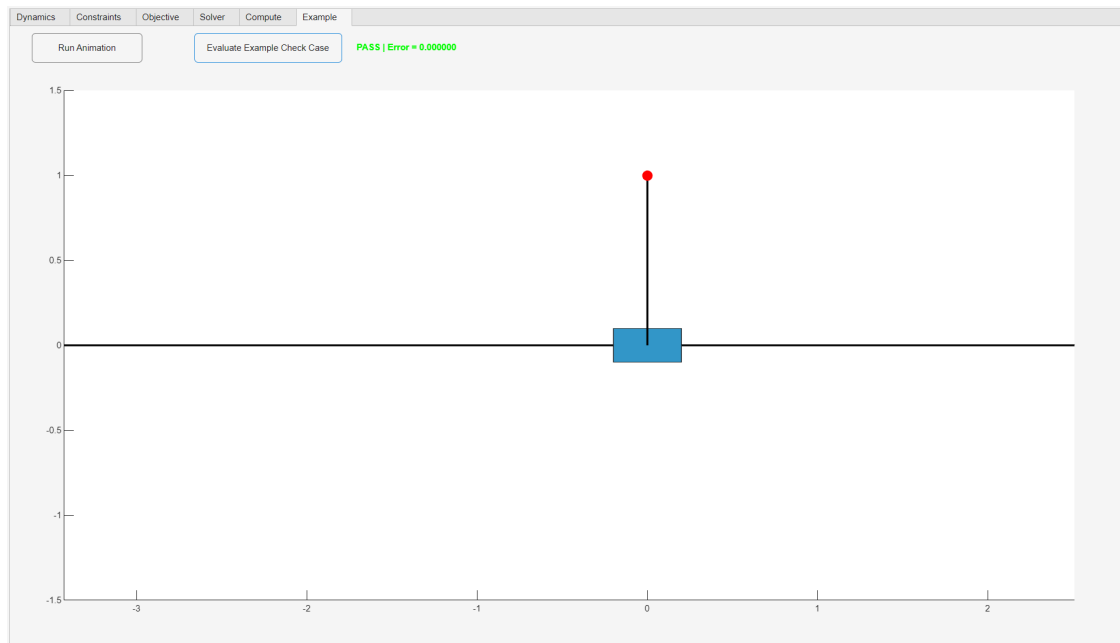


Figure 17: Example Tab of Cart-Pendulum Example